

MODULE 2: ADVANCED DATABASE FEATURES

UNIT 2: TRANSACTION MANAGEMENT AND RECOVERY

2.2.0. Introduction

Transaction processing systems are systems with large databases and hundreds of concurrent users. It provides an “all-or-nothing” proposition stating that each work –unit performed in database must either complete in its entirety or have no effect whatsoever. Users of database systems usually consider consistency and integrity of data as highly important. A simple transaction is usually issued to the database system in a language like SQL wrapped in a transaction. The concepts of Transaction management, types, properties and recovery strategies are described in this unit.

2.2.1. What is a Transaction?

A transaction is a sequence of read and write operations on data items that logically functions as one unit of work.

1. It should either be done entirely or not at all
2. If it succeeds, the effects of write operations persist (commit); if it fails, no effects of write operations persist (abort)
3. These guarantees are made despite concurrent activity in the system, and despite failures that may occur

2.2.1.1 Properties of Transaction

There are four key properties of transactions that a DBMS must ensure to maintain data in the face of concurrent access and system failures:

1. Users should be able to regard the execution of each transaction as **atomic**: either all actions are carried out or none are. Users should not have to worry about the effect of incomplete transactions (say, when a system crash occurs).
2. Each transaction, run by itself with no concurrent execution of other transactions, must preserve the consistency of the database. This property is called **consistency**,

and the DBMS assumes that it holds for each transaction. Ensuring this property of a transaction is the responsibility of the user.

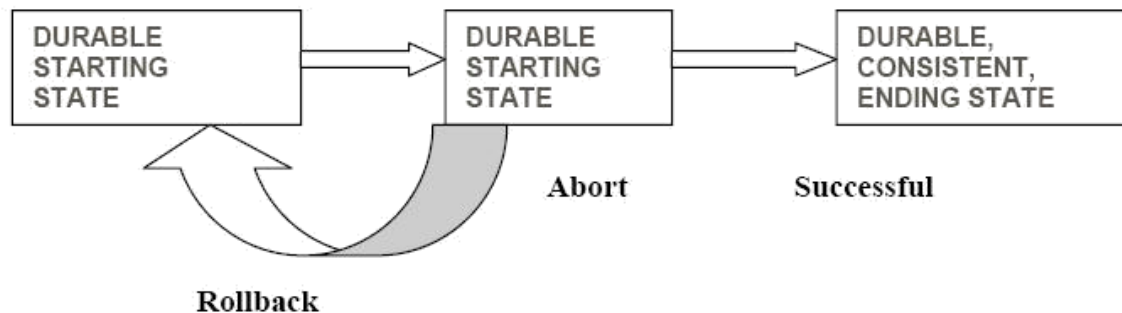


Figure 2.2.1: Database Transaction States

3. Users should be able to understand a transaction without considering the effect of other concurrently executing transactions, even if the DBMS interleaves the actions of several transactions for performance reasons. This property is sometimes referred to as **isolation**: Transactions are isolated, or protected, from the effects of concurrently scheduling other transactions.
4. Once the DBMS informs the user that a transaction has been successfully completed, its effects should persist even if the system crashes before all its changes are reflected on disk. This property is called **durability**.

The acronym **ACID** is sometimes used to refer to the four properties of transactions that we have presented here: atomicity, consistency, isolation and durability. We now consider how each of these properties is ensured in a DBMS.

1. **Consistency and Isolation**: Users are responsible for ensuring transaction consistency. That is, the user who submits a transaction must ensure that when run to completion by itself against a 'consistent' database instance, the transaction will leave the database in a 'consistent' state. For example, the user may have the consistency criterion that fund transfers between bank accounts should not change the total amount of money in the accounts. To transfer money from one account to another, a transaction must debit one account, temporarily leaving the database inconsistent in a global sense, even though the new account balance may satisfy any integrity constraints with respect to the range of acceptable account balances. The

user's notion of a consistent database is preserved when the second account is credited with the transferred amount. If a faulty transfer program always credits the second account with one naira less than the amount debited from the first account, the DBMS cannot be expected to detect inconsistencies due to such errors in the user program's logic.

1. The isolation property is ensured by guaranteeing that even though actions of several transactions might be interleaved, the net effect is identical to executing all transactions one after the other in some serial order. For example, if two transactions T1 and T2 are executed concurrently, the net effect is guaranteed to be equivalent to executing (all of) T1 followed by executing T2 or executing T2 followed by executing T1. (The DBMS provides no guarantees about which of these orders is effectively chosen.) If each transaction maps a consistent database instance to another consistent database instance, executing several transactions one after the other (on a consistent initial database instance) will also result in a consistent final database instance.
2. **Database consistency** is the property that every transaction sees a consistent database instance. Database consistency follows from transaction atomicity, isolation, and transaction consistency. Next, we discuss how atomicity and durability are guaranteed in a DBMS.
2. **Atomicity and Durability:** Transactions can be incomplete for three kinds of reasons. First, a transaction can be aborted, or terminated unsuccessfully, by the DBMS because some anomaly arises during execution. If a transaction is aborted by the DBMS for some internal reason, it is automatically restarted and executed anew. Second, the system may crash (e.g., because the power supply is interrupted) while one or more transactions are in progress. Third, a transaction may encounter an unexpected situation (for example, read an unexpected data value or be unable to access some disk) and decide to abort (i.e., terminate itself). Of course, since users think of transactions as being atomic, a transaction that is interrupted in the middle may leave the database in an inconsistent state. Thus a DBMS must find a way to remove the effects of partial transactions from the database, that is, it must ensure transaction atomicity: either all of a transaction's actions are carried out, or none are. A DBMS ensures transaction atomicity by undoing the actions of incomplete transactions. This means that users can ignore

incomplete transactions in thinking about how the database is modified by transactions over time. To be able to do this, the DBMS maintains a record, called the log, of all writes to the database. The log is also used to ensure durability: If the system crashes before the changes made by a completed transaction are written to disk, the log is used to remember and restore these changes when the system restarts. The DBMS component that ensures atomicity and durability is called the **recovery manager**.

2.2.1.2 Implementing Atomicity

Let's assume that before the transaction took place the balances in the account CA2090 is ₦50,000 and that in the account SB2359 is ₦35000. Now suppose that during the execution of the transaction, a failure (for example, a power failure) occurred that prevented the successful completion of the transaction. The failure occurred after the Write(CA2090); operation was executed, but before the execution of Write(SB2359); in this case the value of the accounts CA2090 and SB2359 are reflected in the database are ₦48,000 and ₦35000 respectively. The ₦2,000 that we have taken from the account is lost. Thus the failure has created a problem. The state of the database no longer reflects a real state of the world that the database is supposed to capture. Such a state is called an **inconsistent state**. The database system should ensure that such inconsistencies are not visible in a database system. It should be noted that even during the successful execution of a transaction there exists points at which the system is in an inconsistent state. But the difference in the case of a successful transaction is that the period for which the database is in an inconsistent state is very short and once the transaction is over the system will be brought back to a consistent state. So if a transaction never started or is completed successfully, the inconsistent states would not be visible except during the execution of the transaction. This is the reason for the atomicity requirement. If the atomicity property provided all actions of the transaction are reflected in the database or none are. The mechanism of maintaining atomicity is as follows The DBMS keeps tracks of the old values of any data on which a transaction performs a Write and if the transaction does not complete its execution, old values are restored to make it appear as though the

transaction never took place. The transaction management component of the DBMS ensures the atomicity of each transaction

2.2.1.23 Implementing Consistencies:

The consistency requirement in the above e. g is that the sum of CA2090 and SB2359 be unchanged by the execution of the transaction. Before the execution of the transaction the amounts in the accounts in CA2090 and SB2359 are ~~₦~~50,000 and ~~₦~~35,000 respectively. After the execution the amounts become ~~₦~~48,000 and ~~₦~~37,000. In both cases the sum of the amounts is 85,000 thus maintaining consistency. Ensuring the consistency for an individual transaction is the responsibility of the application programmer who codes the transaction.

2.2.1.24 Implementing the Isolation

Even if the atomicity and consistency properties are ensured for each transaction there can be problems if several transactions are executed concurrently. The different transactions interfere with one another and cause undesirable results. Suppose we are executing the above transaction T_i . We saw that the database is temporarily inconsistent while the transaction is being executed. Suppose that the transaction has performed the Write(CA2090) operation, during this time another transaction is reading the balances of different accounts. It checks the account CA2090 and finds the account balance at ~~₦~~48,000. Suppose that it reads the account balance of the other account (account SB2359, before the first transaction has got a chance to update the account. So the account balance in the account SB2359 is ~~₦~~35,000. After the second transaction has read the account balances, the first transaction reads the account balance of the account SB2359 and updates it to ~~₦~~37,000. But here we are left with a problem. The first transaction has executed successfully and the database is back to a consistent state. But while it was in an inconsistent state, another transaction performed some operations (May be updated the total account balances). This has left the database in an inconsistent state even after both the transactions have been executed successfully. On solution to the situation (concurrent execution of transactions) is to execute the transactions serially- one after the other. This can create many problems. Suppose long

transactions are being executed first. Then all other transactions will have to wait in the queue. There might be many transactions that are independent (or that do not interfere with one another). There is no need for such transactions to wait in the queue. Also concurrent executions of transactions have significant performance advantages. So the DBMS have found solutions to allow multiple transactions to execute concurrency without any problem. The isolation property of a transaction ensures that the concurrent execution of transactions result in a system state that is equivalent to a state that could have been obtained if the transactions were executed one after another. Ensuring isolation property is the responsibility of the concurrency-control component of the DBMS.

2.2.1.5 Implementing Durability

The durability property guarantees that, once a transaction completes successfully, all updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution. We can guarantee durability by ensuring that either the updates carried out by the transaction have been written to the disk before the transaction completes or information about the updates that are carried out by the transaction and written to the disk are sufficient for the data base to reconstruct the updates when the data base is restarted after the failure. Ensuring durability is the responsibility of the recovery management component of the DBMS. Picture Transaction management and concurrency control components of a DBMS.

2.2.2 Transaction States

Once a transaction is committed, we cannot undo the changes made by the transactions by rolling back the transaction. Only way to undo the effects of a committed transaction is to execute a compensating transaction. The creating of a compensating transaction can be quite complex and so the task is left to the user and it is not handled by the DBMS. The transaction must be in one of the following states:-

1. **Active:-** This is a initial state, the transaction stays in this state while it is executing

2. **Partially committed:** The transaction is in this state when it has executed the final statement
3. **Failed:** A transaction is in this state once the normal execution of the transaction cannot proceed.
4. **Aborted:** A transaction is said to be aborted when the transaction has rolled back and the database is being restored to the consistent state prior to the start of the transaction.
5. **Committed:** a transaction is in this committed state once it has been successfully executed and the database is transformed in to a new consistent state. Different transactions states are given in following figure 2.2.2.

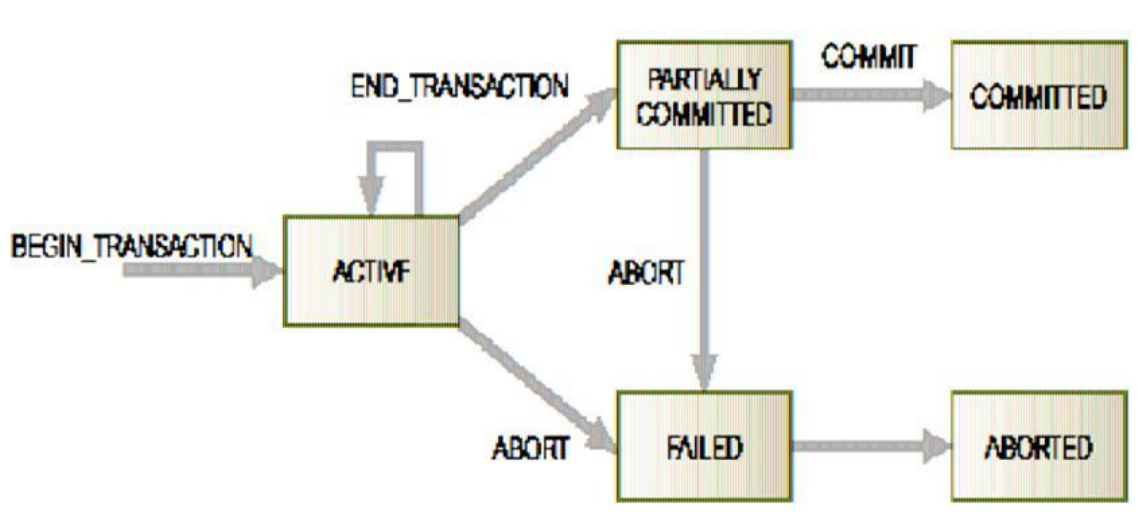


Figure 2.2.2: State Transition Diagram for a Transaction

2.2.3 Transactions and Schedules

A transaction is seen by the DBMS as a series, or list, of actions. The actions that can be executed by a transaction include reads and writes of database objects. A transaction can also be defined as a set of actions that are partially ordered. That is, the relative order of some of the actions may not be important. In order to concentrate on the main issues, we will treat transactions (and later, schedules) as a list of actions. Further, to keep our notation simple, we will assume that an object **O** is always read into a program variable that is also named **O**. We can therefore denote the action of a transaction **T** reading an object **O** as $R_T(O)$; similarly, we can denote writing as $W_T(O)$. When the transaction **T** is clear from the context, we will omit the subscript. In addition to reading and writing, each transaction must specify as its final action either commit (i.e., complete successfully) or abort (i.e., terminate and undo all the actions carried out thus far). **Abort T** denotes the action of **T** aborting, and **Commit T** denotes **T** committing.

A **schedule** is a list of actions (reading, writing, aborting, or committing) from a set of transactions, and the order in which two actions of a transaction **T** appear in a schedule must be the same as the order in which they appear in **T**. Intuitively, a schedule represents an actual or potential execution sequence. For example, the schedule in Table 2.2.1.0 shows an execution order for actions of two transactions **T1** and **T2**. We move forward in time as we go down from one row to the next. We emphasize that a schedule describes the actions of transactions as seen by the DBMS. In addition to these actions, a transaction may carry out other actions, such as reading or writing from operating system files, evaluating arithmetic expressions, and so on.

Table 2.2.1.0: Execution Order for Actions of two transactions

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
R(C)	
W(C)	

Notice that the schedule in Table 2.2.1.0 does not contain an abort or commit action for either transaction. A schedule that contains either an abort or a commit for each transaction whose actions are listed in it is called a **complete schedule**. A complete schedule must contain all the actions of every transaction that appears in it. If the actions of different transactions are not interleaved—that is, transactions are executed from start to finish, one by one—we call the schedule a **serial schedule**.

2.2.4 Concurrent Execution of Transactions

Now that we have introduced the concept of a schedule, we have a convenient way to describe interleaved executions of transactions. The DBMS interleaves the actions of different transactions to improve performance, in terms of increased throughput or improved response times for short transactions, but not all interleaving should be allowed.

2.2.4.1 Motivation for Concurrent Execution

The schedule shown in Table 2.2.1, represents an interleaved execution of the two transactions. Ensuring transaction isolation while permitting such concurrent execution is difficult, but is necessary for performance reasons. First, while one transaction is waiting for a page to be read in from disk, the CPU can process another transaction. This is because I/O activity can be done in parallel with CPU activity in a computer. Overlapping I/O and CPU activity reduces the amount of time disks and processors are idle, and increases **system throughput** (the average number of transactions completed in a given time). Second, interleaved execution of a short transaction with a long transaction usually allows the short transaction to complete quickly. In serial execution, a short transaction could get stuck behind a long transaction leading to unpredictable delays in **response time**, or average time taken to complete a transaction.

Serializability

To begin with, we assume that the database designer has defined some notion of a **consistent database state**. For example, we can define a consistency criterion for a university database to be that the sum of employee salaries in each department should be less than 80 percent of the budget for that department. We require that each transaction must **preserve** database consistency; it follows that any serial schedule that is complete will also preserve database consistency. That is, when a complete serial schedule is executed against a consistent database, the result is also a consistent database.

A **serializable schedule** over a set **S** of committed transactions is a schedule whose effect on any consistent database instance is guaranteed to be identical to that of some complete serial schedule over **S**. That is, the database instance that results from executing the given schedule is identical to the database instance that results from executing the transactions in some serial order.

1. There are some important points to note in this definition: Executing the transactions serially in different orders may produce different results, but all are presumed to be acceptable; the DBMS makes no guarantees about which of them will be the outcome of an interleaved execution.
2. The above definition of a serializable schedule does not cover the case of schedules containing aborted transactions.
3. If a transaction computes a value and prints it to the screen, this is an ‘effect’ that is not directly captured in the state of the database. We will assume that all such values are also written into the database, for simplicity.

2.2.4.1 Anomalies Associated with Interleaved Execution

We now illustrate three main ways in which a schedule involving two consistency preserving, committed transactions could run against a consistent database and leave it in an inconsistent state. Two actions on the same data object **conflict** if at least one of them is a **write**. The three anomalous situations can be described in terms of when the actions of two transactions **T1** and **T2** conflict with each other: in a **write - read (WR)**

conflict $T2$ reads a data object previously written by $T1$; we define **read-write (RW)** and **write-write (WW)** conflicts similarly.

Reading Uncommitted Data (WR Conflicts)

The first source of anomalies is that a transaction $T2$ could read a database object A that has been modified by another transaction $T1$, which has not yet committed. Such a read is called a dirty read. A simple example illustrates how such a schedule could lead to an inconsistent database state. Consider two transactions $T1$ and $T2$, each of which, run alone, preserves database consistency: $T1$ transfers ₦100 from A to B , and $T2$ increments both A and B by 6 percent (e.g., annual interest is deposited into these two accounts). Suppose that their actions are interleaved so that (1) the account transfer program $T1$ deducts ₦100 from account A , then (2) the interest deposit program $T2$ reads the current values of accounts A and B and adds 6 percent interest to each, and then (3) the account transfer program credits ₦100 to account B . The corresponding schedule, which is the view the DBMS has of this series of events, is illustrated in table 2. The result of this schedule is different from any result that

Table 2.2.2: Reading Uncommitted Data

T1	T2
R(A)	
W(A)	
R(B)	R(A)
W(B)	W(A)
Commit	R(B)
	W(B)
	Commit

we would get by running one of the two transactions first and then the other. The problem can be traced to the fact that the value of A written by $T1$ is read by $T2$ before $T1$ has completed all its changes. The general problem illustrated here is that $T1$ may write some value into A that makes the database inconsistent. As long as $T1$ overwrites this value with a ‘correct’ value of A before committing, no harm is done if $T1$ and $T2$ run in some serial order, because $T2$ would then not see the (temporary) inconsistency.

On the other hand, interleaved execution can expose this inconsistency and lead to an inconsistent final database state.

Note that although a transaction must leave a database in a consistent state after it completes, it is not required to keep the database consistent while it is still in progress. Such a requirement would be too restrictive: To transfer money from one account to another, a transaction must debit one account, temporarily leaving the database inconsistent, and then credit the second account, restoring consistency again.

1. **Unrepeatable Reads (RW Conflicts)**

The second way in which anomalous behavior could result is that a transaction **T2** could change the value of an object **A** that has been read by a transaction **T1**, while **T1** is still in progress. This situation causes two problems. First, if **T1** tries to read the value of **A** again, it will get a different result, even though it has not modified **A** in the meantime. This situation could not arise in a serial execution of two transactions; it is called an **unrepeatable read**. Second, suppose that both **T1** and **T2** read the same value of **A**, say, 5, and then **T1**, which wants to increment **A** by 1, changes it to 6, and **T2**, which wants to decrement **A** by 1, decrements the value that it read (that is, 5) and changes **A** to 4. Running these transactions in any serial order should leave **A** with a final value of 5; thus, the interleaved execution leads to an inconsistent state. The underlying problem here is that although **T2**'s change is not directly read by **T1**, it invalidates **T1**'s assumption about the value of **A**, which is the basis for some of **T1**'s subsequent actions.

2. **Overwriting Uncommitted Data (WW Conflicts)**

The third source of anomalous behavior is that a transaction **T2** could overwrite the value of an object **A**, which has already been modified by a transaction **T1**, while **T1** is still in progress. Even if **T2** does not read the value of **A** written by **T1**, a potential problem exists as the following example illustrates. Suppose that Harry and Larry are two employees, and their salaries must be kept equal. Transaction **T1** sets their salaries to ₦1, 000 and transaction **T2** sets their salaries

to ₦2,000. If we execute these in the serial order **T1** followed by **T2**, both receive the salary ₦2,000; the serial order **T2** followed by **T1** gives each the salary ₦1,000. Either of these is acceptable from a consistency standpoint (although Harry and Larry may prefer a higher salary!). Notice that neither transaction reads a salary value before writing it—such a write is called a **blind write**, for obvious reasons. Now, consider the following interleaving of the actions of **T1** and **T2**: **T1** sets Harry's salary to ₦1,000, **T2** sets Larry's salary to ₦2,000, **T1** sets Larry's salary to ₦1,000, and finally **T2** sets Harry's salary to ₦2,000. The result is not identical to the result of either of the two possible serial executions, and the interleaved schedule is therefore not serializable. It violates the desired consistency criterion that the two salaries must be equal.

2.2.4.2 Schedules Involving Aborted Transactions

We now extend our definition of serializability to include aborted transactions. Intuitively, all actions of aborted transactions are to be undone, and we can therefore imagine that they were never carried out to begin with. Using this intuition, we extend the definition of a serializable schedule as follows: A serializable schedule over a set **S** of transactions is a schedule whose effect on any consistent database instance is guaranteed to be identical to that of some complete serial schedule over the set of *committed transactions in S*.

This definition of serializability relies on the actions of aborted transactions being undone completely, which may be impossible in some situations. For example, suppose that (1) an account transfer program **T1** deducts ₦100 from account **A**, then (2) an interest deposit program **T2** reads the current values of accounts **A** and **B** and adds 6 percent interest to each, then commits, and then (3) **T1** is aborted. The corresponding schedule is shown in table 3. Now, **T2** has read a value for **A** that should never have been there (Recall that aborted transactions' effects are not supposed to be visible to other transactions.) If **T2** had not yet committed, we could deal with the situation by cascading the abort of **T1** and also aborting **T2**; this process would recursively abort any transaction that read data written by **T2**, and so on. But **T2** has already committed, and

so we cannot undo its actions. We say that such a schedule is unrecoverable. A **recoverable schedule** is one in which transactions commit only after (and if!) all transactions whose changes they read commit. If transactions read only the changes of committed transactions, not only is the schedule recoverable, but also aborting a transaction can be accomplished without cascading the abort to other transactions. Such a schedule is said to **avoid cascading aborts**. There is another potential problem in undoing the actions of a transaction. Suppose that a transaction **T2** overwrites the value of an object A that has been modified by a transaction **T1**, while **T1** is still in progress, and **T1** subsequently aborts. All of **T1's** changes to database objects are undone by restoring the value of any object that it modified to the value of the object before **T1's** changes. When **T1** is aborted, and its changes are undone in this manner, **T2's** changes are lost as well, even if **T2** decides to commit.

Table 2.2.3: An Unrecoverable Schedule

T1	T2
R(A) W(A)	R(A) W(A) R(B) W(B)
Abort	Commit

So, for example, if **A** originally had the value 5, then was changed by **T1** to 6, and by **T2** to 7, if **T1** now aborts, the value of **A** becomes 5 again. Even if **T2** commits, its change to A is inadvertently lost.

2.2.4.3 LOCK-BASED CONCURRENCY CONTROL

A DBMS must be able to ensure that only serializable, recoverable schedules are allowed, and that no actions of committed transactions are lost while undoing aborted transactions. A DBMS typically uses a locking protocol to achieve this. A locking protocol is a set of rules to be followed by each transaction (and enforced by the

DBMS), in order to ensure that even though actions of several transactions might be interleaved, the net effect is identical to executing all transactions in some serial order.

Strict Two-Phase Locking (Strict 2PL)

The most widely used locking protocol, called Strict Two-Phase Locking, or Strict 2PL, has two rules. The first rule is:

1. If a transaction T wants to read (respectively, modify) an object, it first requests a shared (respectively, exclusive) lock on the object.
2. All locks held by a transaction are released when the transaction is completed. Requests to acquire and release locks can be automatically inserted into transactions by the DBMS; users need not worry about these details.

In effect the locking protocol allows only ‘safe’ interleaving of transactions. If two transactions access completely independent parts of the database, they will be able to concurrently obtain the locks that they need and proceed merrily on their ways. On the other hand, if two transactions access the same object, and one of them wants to modify it, their actions are effectively ordered serially—all actions of one of these transactions (the one that gets the lock on the common object first) are completed before (this lock is released and) the other transaction can proceed. We denote the action of a transaction T requesting a shared (respectively, exclusive) lock on object O as $S_T(O)$ (respectively, $X_T(O)$), and omit the subscript denoting the transaction when it is clear from the context. As an example, consider the schedule shown in table1. This interleaving could result in a state that cannot result from any serial execution of the three transactions. For instance, ***T1*** could change A from 10 to 20, then ***T2*** (which reads the value 20 for A) could change B from 100 to 200, and then $T1$ would read the value 200 for B . If run serially, either ***T1*** or ***T2*** would execute first, and read the values 10 for ***A*** and 100 for ***B***: Clearly, the interleaved execution is not equivalent to either serial execution. If the Strict 2PL protocol is used, the above interleaving is disallowed. Let us see why. Assuming that the transactions proceed at the same relative speed as before, ***T1*** would obtain an exclusive lock on ***A*** first and then read and write A (table 4). Then, ***T2*** would request a lock on A . However, this request cannot be granted until

Table 2.2.5: Schedule Illustrating Strict 2PL

T1	T2
X(A) R(A) W(A)	

T1 releases its exclusive lock on *A*, and the DBMS therefore suspends *T2*. *T1* now proceeds to obtain an exclusive lock on *B*, reads and writes *B*, then finally commits, at which time its locks are released. *T2*'s lock request is now granted, and it proceeds. In this example the locking protocol results in a serial execution of the two transactions, shown in table 5. In general, however, the actions of different transactions could be interleaved. As an example, consider the interleaving of two transactions shown in table 4, which are permitted by the Strict 2PL protocol.

2.2.5 Concurrency control for database management

Concurrency control in database management systems permits many users (assumed to be interactive) to access a database in a multi programmed environment while preserving the illusion that each user has sole access to the system. Control is needed to coordinate concurrent accesses to a DBMS so that the overall correctness of the database is maintained. For example, users *A* and *B* both may wish to read and update the same record in the database at about the same time. The relative timing of the two transactions may have an impact on the state of the database at the end of the transactions. The end result may be an inconsistent database.

Functions of Concurrent Control

Several problems can occur when concurrent transactions execute in an uncontrolled manner.

- The lost update problem: This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of same database item incorrect.
- The temporary update (or dirty read) problem: This occurs when one transaction updates a database item and then the transaction fails for some reason. The

updated item is accessed by another transaction before it is changed back to its original value.

- The incorrect summary problem: If one transaction is calculating an aggregate function on a number of records while other transaction is updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.
1. Whenever a transaction is submitted to a DBMS for execution, the system must make sure that :
 - All the operations in the transaction are completed successfully and their effect is recorded permanently in the database; or
 - the transaction has no effect whatever on the database or on the other transactions in the case of that a transaction fails after executing some of operations but before executing all of them.

Following are the problems created due to the concurrent execution of the transactions:

-Multiple update problems

In this problem, the data written by one transaction (an update operation) is being overwritten by another update transaction. This can be illustrated using our banking example. Consider our account CA2090 that has ₦50,000 balance in it. Suppose a transaction T1 is withdrawing ₦10,000 from the account while another transaction T2 is depositing ₦20,000 to the account. If these transactions were executed serially (one after another), the final balance would be ₦60,000, irrespective of the order in which the transactions are performed. In other words, if the transactions were performed serially, then the result would be the same if T1 is performed first or T2 is performed first-order is not important. But the transactions are performed concurrently, then depending on how the transactions are executed the results will vary. Consider the execution of the transactions given in Table 2.2.6

Table 2.2.6: Execution Transaction

Sequence	T1	T2	Account Balance
01	Begin Transaction		50000
02	Read (CA2090)	Begin Transaction	50000
03	CA2090:=CA2090-10000	Read (CA2090)	50000
04	Write (CA2090)	CA2090:=CA2090+20000	40000
05	Commit	Write (CA2090)	70000
06		Commit	70000

Both transactions start nearly at the same time and both read the account balance of ₦50000. Both transactions perform the operations that they are supposed to perform- T1 will reduce the amount by ₦10000 and will write the result to the data base; T2 will increase the amount by ₦20000 and will write the amount to the database overwriting the previous update. Thus the account balance will gain additional ₦10000 producing a wrong result. If T2 were to start execution first, the result would have been ₦40000 and the result would have been wrong again. This situation could be avoided by preventing T2 from reading the value of the account balance until the update by T1 has been completed.

Incorrect Analysis Problem

Problems could arise even when a transaction is not updating the database. Transactions that read the database can also produce wrong result, if they are allowed to read the database when the database is in an inconsistent state. This problem is often referred to as dirty read or unrepeatable data. The problem of dirty read occurs when a transaction reads several values from the data base while another transactions are updating the values. Consider the case of the transaction that reads the account balances from all accounts to find the total amount in various account. Suppose that there are other transactions, which are updating the account balances-either reducing the amount (withdrawals) or increasing the amount (deposits). So when the first transaction reads the account balances and finds the totals, it will be wrong, as it might have read the account balances before the update in the case of some accounts

and after the updates in other accounts. This problem is solved by preventing the first transaction (the one that reads the balances) from reading the account balances until all the transactions that update the accounts are completed.

Inconsistent Retrievals

Consider two users A and B accessing a department database simultaneously. The user A is updating the database to give all employees a 5% salary raise while user B wants to know the total salary bill of a department. The two transactions interfere since the total salary bill would be changing as the first user updates the employee records. The total salary retrieved by the second user may be a sum of some salaries before the raise and others after the raise. Such a sum could not be considered an acceptable value of the total salary (the value before the raise or after the raise would be).

A	Time	B
Read Employee 100	1	-
-	2	Sum = 0.0
Update Salary	3	-
-	4	Read Employee 100
Write Employee 100	5	-
-	6	Sum = Sum + Salary
Read Employee 101	7	-
-	8	Read Employee 101
Update Salary	9	-
-	10	Sum = Sum + Salary
Write Employee 101	11	-

The problem illustrated in the last example is called the inconsistent retrieval anomaly. During the execution of a transaction therefore, changes made by another transaction that has not yet committed should not be visible since that data may not be consistent.

Uncommitted Dependency

Consider the following situation

A	Time	B
-	1	Read Q
-	2	-
-	3	Write A
Read Q	4	-
-	5	Read R
-	6	-
Write Q	7	-
-	8	Failure (rollback)
-	9	-

Transaction A reads the value of Q that was updated by transaction B but was never committed. The result of Transaction A writing Q therefore will lead to an inconsistent state of the database. Also if the transaction A doesn't write Q but only reads it, it would be using a value of Q which never really existed! Yet another situation would occur if the roll back happens after Q is written by transaction A. The roll back would restore the old value of Q and therefore lead to the loss of updated Q by transaction A. This is called the uncommitted dependency anomaly.

Serializability is a given set of interleaved transactions is said to be serializable if and only if it produces the same results as the serial execution of the same transactions. It is an important concept associated with locking. It guarantees that the work of concurrently executing transactions will leave the database state as it would have been if these transactions had executed serially. This requirement is the ultimate criterion for database consistency and is the motivation for the two-phase locking protocol, which dictates that no new locks can be acquired on behalf of a transaction after the DBMS releases a lock held by that transaction. In practice, this protocol generally means that locks are held until commit time.

Serializability is the classical concurrency scheme. It ensures that a schedule for executing concurrent transactions is equivalent to one that executes the transactions serially in some order. It assumes that all accesses to the database are done using read

and write operations. A schedule is called "correct" if we can find a serial schedule that is "equivalent" to it. Given a set of transactions $T_1 \dots T_n$, two schedules S_1 and S_2 of these transactions are equivalent if the following conditions are satisfied:

1. **Read-Write Synchronization:** If a transaction reads a value written by another transaction in one schedule, then it also does so in the other schedule.
2. **Write-Write Synchronization:** If a transaction overwrites the value of another transaction in one schedule, it also does so in the other schedule.
3. These two properties ensure that there can be no difference in the effects of the two schedules

2.2.5.1 Serializable schedule

A schedule is serial if, for every transaction T participating the schedule, all the operations of T are executed consecutively in the schedule. Otherwise it is called non-serial schedule.

1. Every serial schedule is considered correct; some non-serial schedules give erroneous results.
2. A schedule S of n transactions is serializable if it is equivalent to some serial schedule of the same n transactions; a non-serial schedule which is not equivalent to any serial schedule is not serializable.
3. The definition of two schedules considered "equivalent":— result equivalent: producing same final state of the database (is not used)
 1. **conflict equivalent:** If the order of any two conflicting operations is the same in both schedules.
 2. **view equivalent:** If each read operation of a transaction reads the result of the same write operation in both schedules and the write operations of each transaction must produce the same results.
 3. **Conflict serializable:** if a schedule S is conflict equivalent to some serial schedule. we can reorder the non-conflicting operations S until we form the equivalent serial schedule, and S is a serializable schedule.

4. **View Serializability:** Two schedules are said to be view equivalent if the following three conditions hold. The same set of transactions participate in S and S'; and S and S' include the same operations of those transactions. A schedule S is said to be view serializable if it is view equivalent to a serial schedule.

2.2.5.3 LOCKING

In order to execute transactions in an interleaved manner it is necessary to have some form of concurrency control.

- This enables a more efficient use of computer resources.
- One method of avoiding problems is with the use of locks.
- When a transaction requires a database object it must obtain a lock.

Locking is necessary in a concurrent environment to assure that one process does not retrieve or update a record that is being updated by another process. Failure to use some controls (locking), would result in inconsistent and corrupt data.

Locks enable a multi-user DBMS to maintain the integrity of transactions by isolating a transaction from others executing concurrently. Locks are particularly critical in write intensive and mixed workload (read/write) environments, because they can prevent the inadvertent loss of data or Consistency problems with reads. In addition to record locking, DBMS implements several other locking mechanisms to ensure the integrity of other data structures that provide shared I/O, communication among different processes in a cluster and automatic recovery in the event of a process or cluster failure. Aside from their integrity implications, locks can have a significant impact on performance. While it may benefit a given application to lock a large amount of data (perhaps one or more tables) and hold these locks for a long period of time, doing so inhibits concurrency and increases the likelihood that other applications will have to wait for locked resources.

LOCKING RULES

There are various locking rules that are applicable when a user reads or writes a data to a database. The various locking rules are -

- Any number of transactions can hold S-locks on an item
- If any transaction holds an X-lock on an item, no other transaction may hold any lock on the item
- A transaction holding an X-lock may issue a write or a read request on the data item
- A transaction holding an S-lock may only issue a read request on the data item

2.2.5.4 DEADLOCK

There lies a threat while writing or reading data onto a database with the help of available resources, it is nothing but the deadlock condition. In operating systems or databases, a situation in which two or more processes are prevented from continuing while each waits for resources to be freed by the continuation of the other. Any of a number of situations where two or more processes cannot proceed because they are both waiting for the other to release some resource.

A situation in which processes of a concurrent processor are waiting on an event which will never occur. A simple version of deadlock for a loosely synchronous environment arises when blocking reads and writes are not correctly matched. For example, if two nodes both execute blocking writes to each other at the same time, deadlock will occur since neither write can complete until a complementary read is executed in the other node.

2.2.6 Locking Techniques for Concurrency Control Based on Time Stamp Ordering

The timestamp method for concurrency control does not need any locks and therefore there are no deadlocks. Locking methods generally prevent conflicts by making transaction to wait. Timestamp methods do not make the transactions wait. Transactions involved in a conflict are simply rolled back and restarted. A timestamp is a unique identifier created by the DBMS that indicates the relative starting time of a transaction. Timestamps are generated either using the system clock (generating a timestamp when the transaction starts to execute) or by incrementing a logical counter every time a new transaction starts. Time stamping is the concurrency control protocol in which the fundamental goal is to order transactions globally in such a way that older transactions get priority in the event of a conflict. In the Time stamping method, if a transaction attempts to read or write a data

item, then a read or write operation is allowed only if the last update on that data item was carried out by an older transaction. Otherwise the transaction requesting the read or write is restarted and given a new timestamp to prevent it from continually aborting and restarting. If the restarted transaction is not allowed a new timestamp and is allowed a new timestamp and is allowed to retain the old timestamp, it will never be allowed to perform the read or write, because by that some other transaction which has a newer timestamp than the restarted transaction might not be to commit due to younger transactions having already committed.

In addition to the timestamp for the transactions, data items are also assigned timestamps. Each data item contains a read-timestamp and write-timestamp. The read-timestamp contains the timestamp of the last transaction that read the item and the write-timestamp contains the timestamp of the last transaction that updated the item. For a transaction T the timestamp ordering protocol works as follows:

1. **Transactions T** requests to read the data item 'X' that has already been updated by a younger (later or one with a greater timestamp) transaction. This means that an earlier transactions is trying to read a data item that has been updated by a later transaction T is too late to read the previous outdated value and any other values it has acquired are likely to be inconsistent with the updated value of the data item. In this situation, the transaction T is aborted and restarted with a new timestamp.
2. In all other cases, the transaction is allowed to proceed with the read operation. The read-timestamp of the data item is updated with the timestamp of transaction T.
3. Transaction t requests to write (update) the data item 'X' that has already been read by a younger (later or one with the greater timestamp) transaction. This means that the younger transaction is already using the current value of the data item and it would be an error to update it now. This situation occurs when a transaction is late in performing the write and a younger transaction has already read the old value or written a new one. In this case the transaction T is aborted and is restarted with a new timestamp.

4. Transaction T asks to write the data item 'X' that has already been written by a younger transaction. This means that the transaction T is attempting to write an old or obsolete value of the data item. In this case also the transaction T is aborted and is restarted with a new timestamp.
5. In all other cases the transaction T is allowed to proceed and the write-timestamp of the data item is updated with the timestamp of transaction T.

The above scheme is called basic timestamp ordering. This scheme guarantees that the transactions are conflict serializable and the results are equivalent to a serial schedule in which the transactions are executed in chronological order by the timestamps. In other words, the results of a basic timestamps ordering scheme will be as same as when all the transactions were executed one after another without any interleaving.

One of the problems with basic timestamp ordering is that it does not guarantee recoverable schedules. A modification to the basic timestamp ordering protocol that relaxes the conflict Serializability can be used to provide greater concurrency by rejecting obsolete write operations. This extension is known as Thomas's write rule. Thomas's write rule modifies the checks for a write operation by transaction T as follows.

- When the transaction T requests to write the data item 'X' whose values has already been read by a younger transaction. This means that the order transaction (transaction T) is writing an obsolete value to the data item. In this case the write operation is ignored and the transaction (transaction T) is allowed to continue as if the write were performed. This principle is called the 'ignore obsolete write rule'. This rule allows for greater concurrency.
- In all other cases the transactions T is allowed to proceed and the write timestamp of transaction T.

2.2.6.1 Multiversion Concurrency Control Techniques (Mvcc)

The aim of Multi-Version Concurrency is to avoid the problem of Writers blocking Readers and vice-versa, by making use of multiple versions of data. The problem of

Writers blocking Readers can be avoided if Readers can obtain access to a previous version of the data that is locked by Writers for modification. The problem of Readers blocking Writers can be avoided by ensuring that Readers do not obtain locks on data. Multi-Version Concurrency allows Readers to operate without acquiring any locks, by taking advantage of the fact that if a Writer has updated a particular record, its prior version can be used by the Reader without waiting for the Writer to Commit or Abort. In a Multi-version Concurrency solution, Readers do not block Writers, and vice versa. While Multi-version concurrency improves database concurrency, its impact on data consistency is more complex.

2.2.6.3 Requirements of Multi-Version Concurrency Systems

As its name implies, multi-version concurrency relies upon multiple versions of data to achieve higher levels of concurrency. Typically, a DBMS offering multi-version concurrency (MVDB), needs to provide the following features:

1. The DBMS must be able to retrieve older versions of a row.
2. The DBMS must have a mechanism to determine which version of a row is valid in the context of a transaction. Usually, the DBMS will only consider a version that was committed prior to the start of the transaction that is running the query. In order to determine this, the DBMS must know which transaction created a particular version of a row, and whether this transaction committed prior to the starting of the current transaction.

2.2.6.4 Approaches to Multi-Version Concurrency

There are essentially two approaches to multi-version concurrency. The first approach is to store multiple versions of records in the database, and garbage collect records when they are no longer required. This is the approach adopted by PostgreSQL and Firebird/Interbase. The second approach is to keep only the latest version of data in the database, as in SVDB implementations, but reconstruct older versions of data dynamically as required by exploiting information within the Write Ahead Log. This is the approach taken by Oracle and MySQL/InnoDB.

2.2.6.4 Concept of Database Recovery Management

The **recovery manager** of a DBMS is responsible for ensuring transaction *atomicity* and *durability*. It ensures atomicity by undoing the actions of transactions that do not commit and durability by making sure that all actions of committed transactions survive **system crashes**, (e.g., a core dump caused by a bus error) and **media failures** (e.g., a disk is corrupted). When a DBMS is restarted after crashes, the recovery manager is given control and must bring the database to a consistent state. The recovery manager is also responsible for undoing the actions of an aborted transaction. To see what it takes to implement a recovery manager, it is necessary to understand what happens during normal execution.

Table 2.2.5. Schedule Illustrating Strict 2PL with Serial Execution

T1	T2
X(A)	
R(A)	
W(A)	
X(B)	
R(B)	
W(B)	X(A)
	R(A)
Commit	W(A)
	X(B)
	R(B)
	W(B)
	Commit

Table 2.2.6: Schedule Following Strict 2PL with Interleaved Actions

T1	T2
S(A) R(A)	S(A) R(A) X(B) R(B) W(B)
X(C) R(C) W(C) Commit	Commit

2.2.7 Database Recovery Management

The **transaction manager** of a DBMS controls the execution of transactions. Before reading and writing objects during normal execution, locks must be acquired (and released at some later time) according to a chosen locking protocol. For simplicity of exposition, we make the following assumption.

Atomic Writes: Writing a page to disk is an atomic action. This implies that the system does not crash while a write is in progress and is unrealistic. In practice, disk writes do not have this property, and steps must be taken during restart after a crash to verify that the most recent write to a given page was completed successfully and to deal with the consequences if not.

2.2.7.1 Stealing frames and forcing Pages

With respect to writing objects, two additional questions arise:

1. Can the changes made to an object O in the buffer pool by a transaction T be written to disk before T commits? Such writes are executed when another transaction wants to bring in a page and the buffer manager chooses to replace the page containing O ; of course, this page must have been unpinned by T . If

such writes are allowed, we say that a **steal** approach is used. (Informally, the second transaction 'steals' a frame from T).

2. When a transaction commits, must we ensure that all the changes it has made to objects in the buffer pool are immediately forced to disk? If so, we say that a **force** approach is used.

From the standpoint of implementing a recovery manager, it is simplest to use a buffer manager with a no-steal, force approach. If no-steal is used, we don't have to undo the changes of an aborted transaction (because these changes have not been written to disk), and if force is used, we don't have to redo the changes of a committed transaction. if there is a subsequent crash (because all these changes are guaranteed to have been written to disk at commit time). However, these policies have important drawbacks. The no-steal approach assumes that all pages modified by ongoing transactions can be accommodated in the buffer pool, and in the presence of large transactions (typically run in batch mode, e.g., payroll processing), this assumption is unrealistic. The force approach results in excessive page I/O costs. If a highly used page is updated in succession by 20 transactions, it would be written to disk 20 times. With a no-force approach, on the other hand, the in memory copy of the page would be successively modified and written to disk just once, reflecting the effects of all 20 updates, when the page is eventually replaced in the buffer pool (in accordance with the buffer manager's page replacement policy).

For these reasons, most systems use a steal, no-force approach. Thus, if a frame is dirty and chosen for replacement, the page it contains is written to disk even if the modifying transaction is still active (*steal*); in addition, pages in the buffer pool that are modified by a transaction are not forced to disk when the transaction commits (*no-force*).

2.2.7.1 Recovery-Related steps during normal execution

The recovery manager of a DBMS maintains some information during normal execution of transactions in order to enable it to perform its task in the event of a failure. In particular, a **log** of all modifications to the database is saved on **stable storage**, which is guaranteed (with very high probability) to survive crashes and media failures. Stable

storage is implemented by maintaining multiple copies of information (perhaps in different locations) on nonvolatile storage devices such as disks or tapes. It is important to ensure that the log entries describing a change to the database are written to stable storage *before* the change is made; otherwise, the system might crash just after the change, leaving us without a record of the change.

The log enables the recovery manager to undo the actions of aborted and incomplete transactions and to redo the actions of committed transactions. For example, a transaction that committed before the crash may have made updates to a copy (of a database object) in the buffer pool, and this change may not have been written to disk before the crash, because of a no-force approach. Such changes must be identified using the log, and must be written to disk. Further, changes of transactions that did not commit prior to the crash might have been written to disk because of a steal approach. Such changes must be identified using the log and then undone.

The **recovery manager** of a DBMS is responsible for ensuring two important properties of transactions: *atomicity* and *durability*. It ensures atomicity by undoing the actions of transactions that do not commit and durability by making sure that all actions of committed transactions survive **system crashes**, (e.g., a core dump caused by a bus error) and **media failures** (e.g., a disk is corrupted). The recovery manager is one of the hardest components of a DBMS to design and implement. It must deal with a wide variety of database states because it is called on during system failures. In this unit, we present the **ARIES** recovery algorithm, which is conceptually simple, works well with a wide range of concurrency control mechanisms, and is being used, in an increasing number of database systems. We begin with an introduction to ARIES.

2.2.8 Introduction to ARIES

ARIES is a recovery algorithm that is designed to work with a steal, no-force approach. When the recovery manager is invoked after a crash, restart proceeds in three phases:

1. **Analysis:** Identifies dirty pages in the buffer pool (i.e., changes that have not been written to disk) and active transactions at the time of the crash.

2. **Redo:** Repeats all actions, starting from an appropriate point in the log, and restores the database state to what it was at the time of the crash.
3. **Undo:** Undoes the actions of transactions that did not commit, so that the database reflects only the actions of committed transactions.

There are three main principles behind the ARIES recovery algorithm:

1. **Write-ahead logging:** Any change to a database object is first recorded in the log; the record in the log must be written to stable storage before the change to the database object is written to disk.
2. **Repeating history during Redo:** Upon restart following a crash, ARIES retraces all actions of the DBMS before the crash and brings the system back to the exact state that it was in at the time of the crash. Then, it undoes the actions of transactions that were still active at the time of the crash (effectively aborting them).
3. **Logging changes during Undo:** Changes made to the database while undoing a transaction are logged in order to ensure that such an action is not repeated in the event of repeated (failures causing) restarts.

The second point distinguishes ARIES from other recovery algorithms and is the basis for much of its simplicity and flexibility. In particular, ARIES can support concurrency control protocols that involve locks of finer granularity than a page (e.g., record-level locks). The second and third points are also important in dealing with operations such that redoing and undoing the operation are not exact inverses of each other.

The Log

The log, sometimes called the **trail** or **journal**, is a history of actions executed by the DBMS. Physically, the log is a file of records stored in stable storage, which is assumed to survive crashes; this durability can be achieved by maintaining two or more copies of the log on different disks (perhaps in different locations), so that the chance of all copies of the log being simultaneously lost is negligibly small. The most recent portion of the log, called the **log tail**, is kept in main memory and is periodically forced to stable

storage. This way, log records and data records are written to disk at the same granularity (pages or sets of pages).

1. Every **log record** is given a unique *id* called the **log sequence number (LSN)**. As with any record *id*, we can fetch a log record with one disk access given the LSN. Further, LSNs should be assigned in monotonically increasing order; this property is required for the ARIES recovery algorithm. If the log is a sequential file, in principle growing indefinitely, the LSN can simply be the address of the first byte of the log record. For recovery purposes, every page in the database contains the LSN of the most recent log record that describes a change to this page. This LSN is called the **pageLSN**. A log record is written for each of the following actions:
 1. **Updating a page:** After modifying the page, an *update* type record is appended to the log tail. The pageLSN of the page is then set to the LSN of the update log record. (The page must be pinned in the buffer pool while these actions are carried out.)
 2. **Commit:** When a transaction decides to commit, it **force-writes** a *commit* type log record containing the transaction *id*. That is, the log record is appended to the log, and the log tail is written to stable storage, up to and including the commit record. The transaction is considered to have committed at the instant that its commit log record is written to stable storage.
 3. **Abort:** When a transaction is aborted, an *abort* type log record containing the transaction *id* is appended to the log, and Undo is initiated for this transaction
1. **End:** As noted above, when a transaction is aborted or committed, some additional actions must be taken beyond writing the abort or commit log record. After all these additional steps are completed, an *end* type log record containing the transaction *id* is appended to the log.
2. **Undoing an update:** When a transaction is rolled back (because the transaction is aborted, or during recovery from a crash), its updates are undone. When the action described by an update log record is undone, a *compensation log record*, or CLR, is written.

3. Every log record has certain fields: **prevLSN**, **transID**, and **type**. The set of all log records for a given transaction is maintained as a linked list going back in time, using the **prevLSN** field; this list must be updated whenever a log record is added. The **transID** field is the id of the transaction generating the log record, and the **type** field obviously indicates the type of the log record. Additional fields depend on the type of the log record. We have already mentioned the additional contents of the various log record types, with the exception of the update and compensation log record types, which we describe next.

2.2.8.1 Update Log Records

The fields in an **update** log record are illustrated in table 2.2.7. The **pageID** field

Table 2.2.7: Contents of an Update Log Record

prevLSN	transID	type	pageID	length	offset	before- offset	after- offset
Fields common to all log			Additional fields for update log records				

is the page id of the modified page; the length in bytes and the offset of the change are also included. The **before-image** is the value of the changed bytes before the change; the **after-image** is the value after the change. An update log record that contains both before- and after-images can be used to redo the change and to undo it. In certain contexts, which we will not discuss further, we can recognize that the change will never be undone (or, perhaps, redone). A **redo-only update** log record will contain just the after-image; similarly an **undo-only update** record will contain just the before-image.

Compensation Log Records

A **compensation log record (CLR)** is written just before the change recorded in an update log record *U* is undone. (Such an undo can happen during normal system execution when a transaction is aborted or during recovery from a crash.) A compensation log record *C* describes the action taken to undo the actions recorded in the

corresponding update log record and is appended to the log tail just like any other log record. The compensation log record *C* also contains a field called **undoNextLSN**, which is the LSN of the next log record that is to be undone for the transaction that wrote update record *U*; this field in *C* is set to the value of prevLSN in *U*. As an example, consider the fourth update log record shown in Figure 20.3. If this update is undone, a CLR would be written, and the information in it would include the transID, pageID, length, offset, and before-image fields from the update record. Notice that the CLR records the (undo) action of changing the affected bytes back to the before-image value; thus, this value and the location of the affected bytes constitute the redo information for the action described by the CLR.

Unlike an update log record, a CLR describes an action that will never be *undone*, that is, we never undo an undo action. The reason is simple: an update log record describes a change made by a transaction during normal execution and the transaction may subsequently be aborted, whereas a CLR describes an action taken to rollback a transaction for which the decision to abort has already been made. Thus, the transaction *must* be rolled back, and the undo action described by the CLR is definitely required. This observation is very useful because it bounds the amount of space needed for the log during restart from a crash: The number of CLRs that can be written during Undo is no more than the number of update log records for active transactions at the time of the crash. It may well happen that a CLR is written to stable storage (following WAL, of course) but that the undo action that it describes is not yet written to disk when the system crashes again. In this case the undo action described in the CLR is reapplied during the Redo phase, just like the action described in update log records. For these reasons, a CLR contains the information needed to reapply, or redo, the change described but not to reverse it.

2.2.8.3 Other Recovery-Related Data Structures

In addition to the log, the following two tables contain important recovery-related information:

1. **Transaction table:** This table contains one entry for each active transaction. The entry contains (among other things) the transaction id, the status, and a field called **lastLSN**, which is the LSN of the most recent log record for this transaction. The **status** of a transaction can be that it is in progress, is committed, or is aborted. (In the latter two cases, the transaction will be removed from the table once certain 'clean up' steps are completed.)

Dirty page table: This table contains one entry for each dirty page in the buffer pool, that is, each page with changes that are not yet reflected on disk. The entry contains a field **recLSN**, which is the LSN of the first log record that caused the page to become dirty. Note that this LSN identifies the earliest log record that might have to be redone for this page during restart from a crash. During normal operation, these are maintained by the transaction manager and the buffer manager, respectively, and during restart after a crash, these tables are reconstructed in the Analysis phase of restart.

The Write-Ahead Log Protocol

Before writing a page to disk, every update log record that describes a change to this page must be forced to stable storage. This is accomplished by forcing all log records up to and including the one with LSN equal to the pageLSN to stable storage before writing the page to disk.

The importance of the WAL protocol cannot be over emphasized WAL is the fundamental rule that ensures that a record of every change to the database is available while attempting to recover from a crash. If a transaction made a change and committed, the no-force approach means that some of these changes may not have been written to disk at the time of a subsequent crash. Without a record of these changes, there would be no way to ensure that the changes of a committed transaction survive crashes. Note that the definition of a *committed transaction* is effectively "a transaction whose log records, including a commit record, have all been written to stable storage"! When a transaction is committed, the log tail is forced to stable storage, even if a no force approach is being used. It is worth contrasting this operation with the actions taken

under a force approach: If a force approach is used, all the pages modified by the transaction, rather than a portion of the log that includes all its records, must be forced to disk when the transaction commits.

The set of all changed pages is typically much larger than the log tail because the size of an update log record is close to (twice) the size of the changed bytes, which is likely to be much smaller than the page size. Further, the log is maintained as a sequential file, and thus all writes to the log are sequential writes. Consequently, the cost of forcing the log tail is much smaller than the cost of writing all changed pages to disk.

Checkpointing

A **checkpoint** is like a snapshot of the DBMS state, and by taking checkpoints periodically, as we will see, the DBMS can reduce the amount of work to be done during restart in the event of a subsequent crash. *Checkpointing* in ARIES has three steps. First, a **begin checkpoint** record is written to indicate when the checkpoint starts. Second, an **end checkpoint** record is constructed, including in it the current contents of the transaction table and the dirty page table, and appended to the log. The third step is carried out after the **end checkpoint** record is written to stable storage: A special **master** record containing the LSN of the *begin checkpoint* log record is written to a known place on stable storage. While the end checkpoint record is being constructed, the DBMS continues executing transactions and writing other log records; the only guarantee we have is that the transaction table and dirty page table are accurate *as of the time of the begin checkpoint record*.

This kind of checkpoint is called a **fuzzy checkpoint** and is inexpensive because it does not require quiescing the system or writing out pages in the buffer pool (unlike some other forms of checkpointing). On the other hand, the effectiveness of this checkpointing technique is limited by the earliest recLSN of pages in the dirty pages table, because during restart we must redo changes starting from the log record whose LSN is equal to this recLSN. Having a background process that periodically writes dirty

pages to disk helps to limit this problem. When the system comes back up after a crash, the restart process begins by locating the most recent checkpoint record. For uniformity, the system always begins normal execution by taking a checkpoint, in which the transaction table and dirty page table are both empty.

2.2.8.4 Recovering from a System Crash

When the system is restarted after a crash, the recovery manager proceeds in three Phases. The Analysis phase begins by examining the most recent begin checkpoint record, whose LSN is denoted as C and proceeds forward in the log until the last log record. The Redo phase follows Analysis and redoes all changes to any page that might have been dirty at the time of the crash; this set of pages and the starting point for Redo (the smallest recLSN of any dirty page) are determined during Analysis. The Undo phase follows Redo and undoes the changes of all transactions that were active at the time of the crash; again, this set of transactions is identified during the Analysis phase. Notice that Redo reapplies changes in the order in which they were originally carried out; Undo reverses changes in the opposite order, reversing the most recent change first.

Analysis Phase

The **Analysis** phase performs three tasks:

2. It determines the point in the log at which to start the Redo pass.
3. It determines (a conservative superset of the) pages in the buffer pool that were dirty at the time of the crash.
4. It identifies transactions that were active at the time of the crash and must be undone.

Analysis begins by examining the most recent begin checkpoint log record and initializing the dirty page table and transaction table to the copies of those structures in the next end checkpoint record. Thus, these tables are initialized to the set of dirty pages and active transactions at the time of the checkpoint. (If there are additional log records between the begin checkpoint and end checkpoint records, the tables must be adjusted to

reflect the information in these records, but we omit the details of this Analysis then scans the log in the forward direction until it reaches the end of the log:

1. If an end log record for a transaction T is encountered, T is removed from the transaction table because it is no longer active.
2. If a log record other than an end record for a transaction T is encountered, an entry for T is added to the transaction table if it is not already there. Further, the entry for T is modified:
 1. The lastLSN field is set to the LSN of this log record.
 2. If the log record is a commit record, the status is set to C, otherwise it is set to U (indicating that it is to be undone).
1. If a redo able log record affecting page P is encountered, and P is not in the dirty page table, an entry is inserted into this table with page id P and recLSN equal to the LSN of this redoable log record. This LSN identifies the oldest change affecting page P that may not have been written to disk.

At the end of the Analysis phase, the transaction table contains an accurate list of all transactions that were active at the time of the crash | this is the set of transactions with status U. The dirty page table includes all pages that were dirty at the time of the crash, but may also contain some pages that were written to disk. If an *end write* log record were written at the completion of each write operation, the dirty page table constructed during Analysis could be made more accurate, but in ARIES, the additional cost of writing end write log records is not considered to be worth the gain.

2.2.8.4 Redo Phase

1. During the **Redo** phase, ARIES reapplies the updates of *all* transactions, committed or otherwise. Further, if a transaction was aborted before the crash and its updates were undone, as indicated by CLRs, the actions described in the CLRs are also reapplied.

2. This **repeating history** paradigm distinguishes ARIES from other proposed WALbased recovery algorithms and causes the database to be brought to the same state that it was in at the time of the crash.
3. The Redo phase begins with the log record that has the smallest recLSN of all pages in the dirty page table constructed by the Analysis pass because this log record identifies the oldest update that may not have been written to disk prior to the crash. Starting from this log record, Redo scans forward until the end of the log. For each redo able log record (update or CLR) encountered, Redo checks whether the logged action must be redone. The action must be redone unless one of the following conditions holds:
 1. The affected page is not in the dirty page table, or
 2. The affected page is in the dirty page table, but the recLSN for the entry is *greater than* the LSN of the log record being checked, or
 3. The pageLSN (stored on the page, which must be retrieved to check this condition) is *greater than or equal* to the LSN of the log record being checked.
 1. The first condition obviously means that all changes to this page have been written to disk. Because the recLSN is the first update to this page that may not have been written to disk, the second condition means that the update being checked was indeed propagated to disk. The third condition, which is checked last because it requires us to retrieve the page, also ensures that the update being checked was written to disk because either this update or a later update to the page was written. (Recall our assumption that a write to a page is atomic; this assumption is important here!) If the logged action must be redone:
 2. The logged action is reapplied.
 3. The pageLSN on the page is set to the LSN of the redone log record. No additional log record is written at this time.

2.2.8.5 Undo Phase

The Undo phase, unlike the other two phases, scans backward from the end of the log. The goal of this phase is to undo the actions of all transactions that were active at the time of the crash, that is, to effectively abort them. This set of transactions is identified in the transaction table constructed by the Analysis phase.

The Undo Algorithm

Undo begins with the transaction table constructed by the Analysis phase, which identifies all transactions that were active at the time of the crash, and includes the LSN of the most recent log record (the lastLSN field) for each such transaction. Such transactions are called **loser transactions**. All actions of losers must be undone, and further, these actions must be undone in the reverse of the order in which they appear in the log.

Consider the set of lastLSN values for all loser transactions. Let us call this set **ToUndo**. Undo repeatedly chooses the largest (i.e., most recent) LSN value in this set and processes it, until ToUndo is empty. To process a log record:

1. If it is a CLR, and the undoNextLSN value is not *null*, the undoNextLSN value is added to the set ToUndo; if the undoNextLSN is *null*, an end record is written for the transaction because it is completely undone, and the CLR is discarded.
2. If it is an update record, a CLR is written and the corresponding action is undone and the prevLSN value in the update log record is added to the set ToUndo. When the set ToUndo is empty, the Undo phase is complete. Restart is now complete, and the system can proceed with normal operations.

Aborting a Transaction

Aborting a transaction is just a special case of the Undo phase of Restart in which a single transaction, rather than a set of transactions is undone.

2.2.9 Media Recovery

Media recovery is based on periodically making a copy of the database. Because copying a large database object such as a file can take a long time, and the DBMS must be allowed to continue with its operations in the meantime, creating a copy is handled in a manner similar to taking a fuzzy checkpoint.

When a database object such as a file or a page is corrupted, the copy of that object is brought up-to-date by using the log to identify and reapply the changes of committed transactions and undo the changes of uncommitted transactions (as of the time of the media recovery operation). The begin checkpoint LSN of the most recent complete checkpoint is recorded along with the copy of the database object in order to minimize the work in reapplying changes of committed transactions. Let us compare the smallest recLSN of a dirty page in the corresponding end checkpoint record with the LSN of the begin checkpoint record and call the smaller of these two LSNs I .

We observe that the actions recorded in all log records with LSNs less than I must be reflected in the copy. Thus, only log records with LSNs greater than I need to be reapplied to the copy. Finally, the updates of transactions that are incomplete at the time of media recovery or that were aborted after the fuzzy copy was completed need to be undone to ensure that the page reflects only the actions of committed transactions. The set of such transactions can be identified as in the Analysis pass.

2.2.9.1 Shadow Paging

This recovery scheme does not require the use of a log in a single-user environment. In a multiuser environment, a log may be needed for the concurrency control method. Shadow paging considers the database to be made up of a number of fixed-size disk pages (or disk blocks)-say, n -for recovery purposes. A directory with n entries' is constructed, where the i^{th} entry points to the i^{th} database page on disk. The directory is kept in main memory if it is not too large, and all references-reads or writes-to database pages on disk go through it. When a transaction begins executing, the current directory-whose entries point to the most recent or current database pages on disk-is copied into a shadow directory. The shadow directory is then saved on disk while the current directory is used by the transaction. During transaction execution, the shadow directory

is *never* modified. When a write_ item operation is performed, a new copy of the modified database page is created, but the old copy of that page is *not overwritten*. Instead, the new page is written elsewhere-on some previously unused disk block. The current directory entry is modified to point to the new disk block, whereas the shadow directory is not modified and continues to point to the old unmodified disk block

Committing a transaction corresponds to discarding the previous shadow directory. Since recovery involves neither undoing nor redoing data items, this technique can be categorized as a NO-UNDO/NO-REDO technique for recovery. In a multiuser environment with concurrent transactions, logs and checkpoints must be incorporated into the shadow paging technique. One disadvantage of shadow paging is that the updated database pages change location on disk. This makes it difficult to keep related database pages close together on disk without complex storage management strategies. Furthermore, if the directory is large, the overhead of writing shadow directories to disk as transactions commit is significant. A further complication is how to handle garbage collection when a transaction commits. The old pages referenced by the shadow directory that have been updated must be released and added to a list of free pages for future use. These pages are no longer needed after the transaction commits. Another issue is that the operation to migrate between current and shadow directories must be implemented as an atomic operation.

2.2.9.2 Other algorithms and interaction with Concurrency Control

Like ARIES, the most popular alternative recovery algorithms also maintain a log of database actions according to the WAL protocol. A major distinction between ARIES and these variants is that the Redo phase in ARIES *repeats history*, that is, redoes the actions of *all* transactions, not just the non-losers. Other algorithms redo only the non-losers, and the Redo phase follows the Undo phase, in which the actions of losers are rolled back.

Repeating history paradigm and the use of CLRs, ARIES is able to support fine-granularity locks (record-level locks) and logging of logical operations, rather than just byte-level modifications. For example, consider a transaction *T* that inserts a data entry 15* into a B+ tree index. Between the times this insert is done and the time that *T* is

eventually aborted, other transactions may also insert and delete entries from the tree. If record-level locks are set, rather than page-level locks, it is possible that the entry 15* is on a different physical page when T aborts from the one that T inserted it into. In this case the undo operation for the insert of 15* must be recorded in logical terms because the physical (byte-level) actions involved in undoing this operation are not the inverse of the physical actions involved in inserting the entry.

Logging logical operations yields considerably higher concurrency, although the use of fine-granularity locks can lead to increased locking activity (because more locks must be set). Thus, there is a trade-off between different WAL-based recovery schemes. We have chosen to cover ARIES because it has several attractive properties, in particular, its simplicity and its ability to support fine-granularity locks and logging of logical operations.

One of the earliest recovery algorithms, used in the System R prototype at IBM, takes a very different approach. There is no logging and, of course, no WAL protocol. Instead, the database is treated as a collection of pages and accessed through a **page table**, which maps page ids to disk addresses. When a transaction makes changes to a data page, it actually makes a copy of the page, called the **shadow** of the page, and changes the shadow page. The transaction copies the appropriate part of the page table and changes the entry for the changed page to point to the shadow, so that it can see the changes; however, other transactions continue to see the original page table, and therefore the original page, until this transaction commits. Aborting a transaction is simple: just discard its shadow versions of the page table and the data pages. Committing a transaction involves making its version of the page table public and discarding the original data pages that are superseded by shadow pages.

This scheme suffers from a number of problems. First, data becomes highly fragmented due to the replacement of pages by shadow versions, which may be located far from the original page. This phenomenon reduces data clustering and makes good garbage collection imperative. Second, the scheme does not yield a sufficiently high degree of concurrency. Third, there is a substantial storage overhead due to the use of

shadow pages. Fourth, the process aborting a transaction can itself run into deadlocks, and this situation must be specially handled because the semantics of aborting an abort transaction gets murky. For these reasons, even in System R, shadow paging was eventually superseded by WAL-based recovery techniques.

2.2.9.3 Database Backup and Recovery from Catastrophic Failures

So far, all the techniques we have discussed apply to non-catastrophic failures. A key assumption has been that the system log is maintained on the disk and is not lost as a result of the failure. Similarly, the shadow directory must be stored on disk to allow recovery when shadow paging is used. The recovery techniques we have discussed use the entries in the system log or the shadow directory to recover from failure by bringing the database back to a consistent state.

The recovery manager of a DBMS must also be equipped to handle more catastrophic failures such as disk crashes. The main technique used to handle such crashes is that of database backup. The whole database and the log are periodically copied onto a cheap storage medium such as magnetic tapes. In case of a catastrophic system failure, the latest backup copy can be reloaded from the tape to the disk, and the system can be restarted. To avoid losing all the effects of transactions that have been executed since the last backup, it is customary to back up the system log at more frequent intervals than full database backup by periodically copying it to magnetic tape. The system log is usually substantially smaller than the database itself and hence can be backed up more frequently. Thus users do not lose all transactions they have performed since the last database backup. All committed transactions recorded in the portion of the system log that has been backed up to tape can have their effect on the database redone. A new log is started after each database backup. Hence, to recover from disk failure, the database is first recreated on disk from its latest backup copy on tape.

1. Self-Assessment Exercises

1. How does the recovery manager ensure atomicity of transactions?
2. How does it ensure durability?
3. What is the difference between stable storage and disk?
4. What is the difference between a system crash and a media failure?
5. Explain the WAL protocol.
6. Describe the steal and no-force policies

2.2.11 Tutor Marked Assignment

1. What are the properties required of LSNs?
2. What are the fields in an update log record? Explain the use of each field.
3. What are redo able log records?
4. What are the differences between update log records and CLRs?

2.2.12 References/Suggested Readings

- 1 Date, C.J., Introduction to Database Systems (7th Edition) Addison Wesley, 2000
- 2 Leon, Alexis and Leon, Mathews, Database Management Systems, LeonTECHWorld
- 3 Elamasri R . and Navathe, S., Fundamentals of Database Systems (3 Edition), Pearson Education, 2000.